Bhagiratha: AI-based Village Pond Planning System

CSD Assignment 1 – Final Technical Report

RAHUL SEERA, 12341980, Indian Institute of Technology Bhilai, India

Village ponds are often dug where land happens to be free rather than where rainwater collects, so many stay dry or breach in the monsoon. Bhagiratha is a web application that recommends where a pond should go and how large it should be. An administrator clicks a point, draws the available land on a satellite map, or uploads a contour map (KML/KMZ). The backend builds an elevation grid (SRTM 30 m from OpenTopography, or Delaunay interpolation of the contours), fills artificial depressions with Priority-Flood, routes flow with a latitude-corrected D8 algorithm and picks the cell where the most runoff converges, restricted to the drawn parcel in land-area mode. It delineates that point's catchment, takes the 10-year June–September rainfall from the Open-Meteo archive, estimates runoff as $Q = C \cdot I \cdot A$ with $C = 0.35$, and sizes a pond 2.5–4.5 m deep holding 70% of Q, with a 0–100 suitability score. The pond site, catchment boundary, catchment area and expected water volume are overlaid on the map. On a real 1 m contour survey near Durg (1,355 contours) the system finds a 3.9116 km^2 catchment and 1,713,242 m^3 of monsoon runoff in under 1 s. It is deployed on a campus container limited to one CPU and 512 MiB of RAM; stress tests there (446 requests, up to 20 concurrent) completed with zero errors and a peak memory of 330 MB after admission control was added to stop out-of-memory crashes.

Additional Key Words and Phrases: Village Pond Planning, Geospatial Analysis, Rainwater Harvesting, Catchment Estimation, Web GIS, REST APIs

Live application: http://10.1.75.53:3246/ (IIT Bhilai campus network)

GitHub repository: https://github.com/RahulSeera/pond-planning-telemetry

Demo video: https://youtu.be/vMweMjVhEAE

1 Introduction

Most rural Indian agriculture depends on the June–September monsoon. Storing that runoff in village ponds (*Amrit Sarovars*) recharges groundwater and carries irrigation and livestock water into the dry months. A pond only works, though, if it sits where runoff naturally converges and is sized to the water that actually arrives. This project builds a web application that recommends such sites from terrain and historical rainfall, and shows the pond location, catchment boundary, catchment area and expected water volume on an interactive map. Section 2 maps the requirements to modules, §3 presents the architecture, §4 the algorithms, §5 the implementation, §6 the CSD themes, §7 the results and load tests, and §8 the limitations.

1.1 Motivation

Manual siting is hard for three reasons. The land that drains to a candidate point can span several square kilometres and cannot be traced reliably by eye. Rainfall records are scattered, so ponds are rarely sized to measured rainfall. And the decision is usually driven by which plot is free, not by hydrology. An automated tool gives a village administrator a defensible first answer in seconds and restricts the search to land they know is available.

1.2 Scope of the Project

In scope: satellite and topographic map display; contour-map ingestion (KML/KMZ); selection of a land parcel on the map; automatic choice of the maximum-runoff-convergence point (inside the parcel, or over a contour map); evaluation of a user-clicked point; catchment delineation and area; 10-year rainfall; seasonal runoff volume; pond depth, surface

area, capacity and a suitability score; map overlay and GeoJSON export. **Out of scope:** the tool does not know about existing rivers, canals or tanks, land ownership or encroachment, land use and buildings, soil permeability, groundwater, flood safety, downstream water rights, approvals or cost, and it does not design the embankment or spillway. Its output is a hydrological screening recommendation that must be verified in the field.

2 Problem Statement and Requirements

The system must let a user select land on a map and return, for that land, a suggested pond location, the catchment area draining to it and the expected water volume, all overlaid on the map, quickly and reliably within the provided server. Table 1 maps each functional requirement to the code that satisfies it.

Table 1. Functional requirements and where they are implemented

Requirement	Implemented in
Satellite imagery display	`static/js/app.js` (`initMap`: Esri World Imagery + topographic layers)
Contour map visualization	`kml_parser.py`, `dem_from_contours.py` (KML/KMZ → elevation grid)
Available-land identification	`app.js` (Draw Area Box), `area_analysis.py` (site search inside the parcel)
Catchment area estimation	`catchment.py` (`find_pond_site`, `delineate_catchment`)
Historical rainfall query	`rainfall.py` (Open-Meteo 10-year archive, bundled fallback)
Runoff volume estimation	`recommendation.py` ($Q = C \cdot I \cdot A$)
Pond depth / storage recommendation	`recommendation.py` (depth, area, capacity, suitability score)
Combined overlay / results view	`app.js` (`renderMapOverlays`, `renderResults`)

2.1 Non-Functional Requirements

- **Response time:** contour analysis under 3 s; repeated queries under 1 s; point and land-area analysis under 30 s (dominated by the elevation download).
- **Concurrency and resources:** tolerate at least 20 concurrent users on the provided server (one CPU, 512 MiB RAM) without errors or crashes; heavy jobs may queue, but must not overload memory.
- **Availability:** the service restarts itself within 60 s of a failure, and never returns HTTP 500 because an optional dependency (database, rainfall or geocoding API) is unavailable.
- **Usability:** a single page on desktop Chrome and Firefox; every degraded result is explained on screen.
- **Security:** validated inputs (schema, file type), no secrets in the repository (API keys in `.env`); no authentication, as the tool performs read-only analysis on the campus network.

3 System Architecture and High-Level Design

Bhagiratha is an asynchronous *modular monolith*: one FastAPI process with engines separated by typed interfaces (Pydantic models in and out). Figure 1 shows the layers. The single-page client renders the map and results; the API layer validates requests, consults a response cache and orchestrates the engines; the domain engines compute terrain, rainfall and the recommendation; persistence is PostGIS when available and JSON files otherwise; two shell scripts supervise the process. Data flows from a map action to the API, out to OpenTopography and Open-Meteo, through the engines, into storage, and back to the map as JSON plus a GeoJSON polygon.

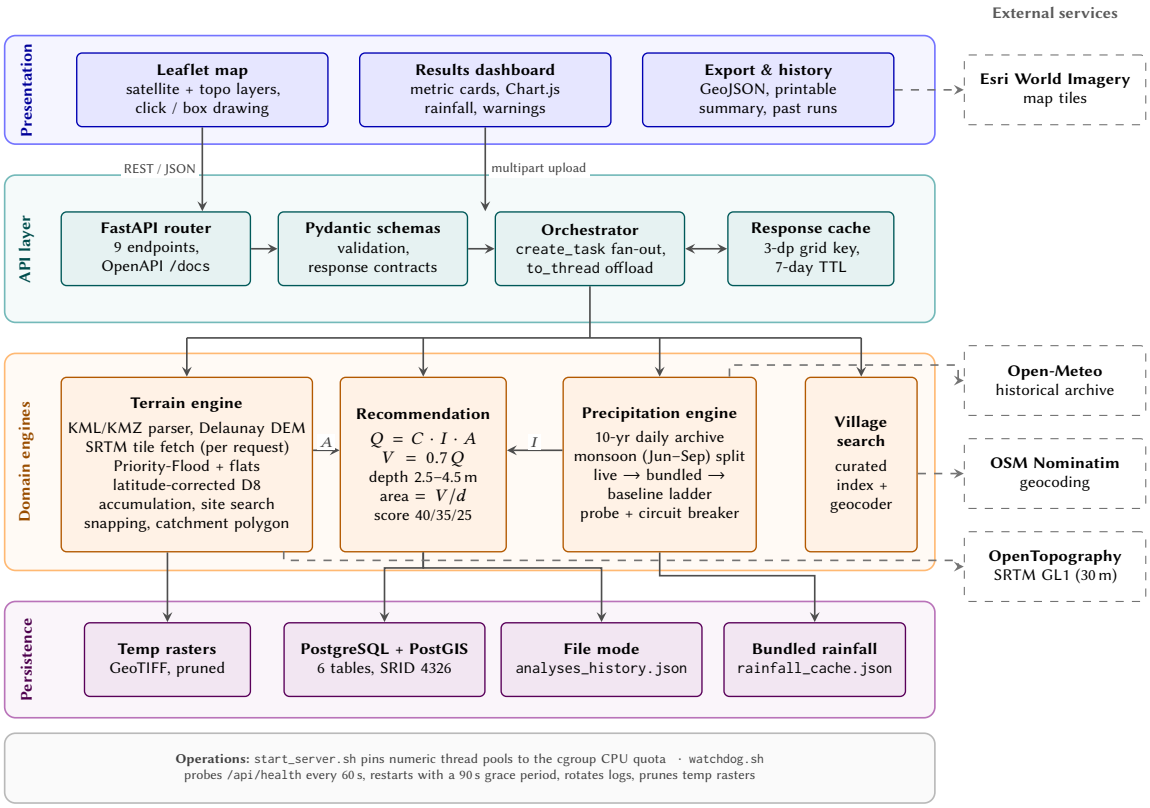


Fig. 1. Layered component architecture. A: catchment area and slope; I: monsoon rainfall depth. Solid arrows are in-process calls; dashed arrows are network calls to external services, each backed by a local fallback (§6).

3.1 Technology Stack

Python 3.12 with FastAPI, Uvicorn and Pydantic; Rasterio, PySheds, NumPy and SciPy for terrain analysis; lxml for KML; httpx for asynchronous API calls; PostgreSQL + PostGIS via SQLAlchemy and GeoAlchemy2, with a JSON-file fallback; a vanilla-JavaScript frontend with Leaflet.js (Esri World Imagery) and Chart.js. External data: OpenTopography (SRTM GL1, 30 m), the Open-Meteo historical archive and OpenStreetMap Nominatim. *Deviation from the brief:* OpenCV was not used. The terrain work is raster hydrology (depression filling, flow direction, accumulation), for which PySheds provides tested implementations of the standard algorithms; image-processing primitives would have had to re-implement them.

4 Methodology

4.1 Terrain and Elevation Analysis

Two elevation sources are supported. For a clicked point or drawn parcel, an SRTM GL1 GeoTIFF is downloaded from OpenTopography covering the area plus a margin, so the upstream catchment is not cut off. For an uploaded contour map, every vertex of every `LineString` becomes an elevation sample (elevation from the placemark name, or the mean z); dense inputs are decimated with a deterministic stride (159,113 → 22,731 points for the sample file), and `scipy.interpolate.griddata` interpolates them linearly on a Delaunay triangulation onto a 152×200 grid, with

nearest-neighbour fill outside the convex hull. The grid is written as a GeoTIFF whose pixel centres coincide with the sample points. Both sources are normalised to `float32` and processed by the same pipeline.

4.2 Catchment Area Delineation

(1) **Conditioning:** Priority-Flood [1] raises every closed depression to its spill level ($O(N \log N)$), and flats receive a small gradient so every cell drains.
(2) **D8 flow direction [2]:** each cell drains to the neighbour with the steepest descent $(z_{i,j} - z_k)/d_k$. The raster is in degrees, so d_k uses metric cell sizes $\Delta x = \Delta\lambda \cdot 111{,}320 \cos\phi$ and $\Delta y = \Delta\phi \cdot 111{,}320$; without this, east–west slopes are misjudged by about 7% at 21°N.
(3) **Flow accumulation** ($O(N)$) counts the cells draining through each cell.
(4) **Pond site:** contour and land-area modes choose arg max accumulation over the interior, excluding an 8% border where edge effects distort flow; land-area mode further restricts the search to the drawn parcel. Point mode instead evaluates the clicked location, snapped to the highest-accumulation cell within 2 cells (≈60 m), as in standard GIS practice.
(5) **Catchment:** the D8 tree is traced upstream from the site, polygonized to GeoJSON, and its area is the cell count times the metric cell area. If the catchment touches the raster edge, a larger tile is fetched once, and the area is otherwise reported as a lower bound.

Assumptions: surface flow follows the steepest descent on a 30 m (SRTM) or interpolated grid; "best site" means maximum runoff convergence and nothing else.

4.3 Rainfall Data Integration

Daily `precipitation_sum` for the pond site is requested from the Open-Meteo historical archive for the last ten complete years (2016–2025), aggregated into mean annual and mean June–September (monsoon) totals. The campus server cannot reach Open-Meteo, so real archive statistics for 19 points around Durg–Bhilai are bundled with the app (`data/rainfall_cache.json`, built by a script from the same API) and the nearest point within 11 km is used. Elsewhere a conservative regional baseline (1,150 mm/yr, 920 mm monsoon) is used, and the user is always told which source applied.

4.4 Runoff Estimation and Pond Sizing

Seasonal runoff uses the volumetric form of the Rational Method [3]: $Q = C \cdot I \cdot A$, with A the catchment area (m²), I the mean monsoon depth (m) and $C = 0.35$, a typical coefficient for cultivated loamy land. The pond holds $V = 0.70\,Q$ (the remainder spills). Depth rises linearly from 2.5 m (for $V \le 5{,}000$ m³; shallower ponds lose proportionally more to evaporation) to 4.5 m (for $V \ge 200{,}000$ m³; the practical limit for unlined earthen banks), and surface area is V/d. The suitability score is $0.40\,S_{catchment} + 0.35\,S_{slope} + 0.25\,S_{rain}$, clipped to $[5, 98]$, with piecewise-linear sub-scores: full marks for catchments of 0.5–5 km², slopes up to 4% and rainfall of at least 1,000 mm.

5 Implementation

5.1 Backend and API Design

Table 2 lists the endpoints; full OpenAPI documentation is served at `/docs`. Invalid bodies return 422, unsupported or unparseable files 400, a missing elevation tile 502, and a full analysis queue 503 with `Retry-After`. Third-party failures

are handled by fallbacks (bundled rainfall, then baseline; curated village list; one retry for elevation downloads) that are always reported in the response's `warnings` list; a background probe and a circuit breaker keep blocked services off the request path.

Table 2. REST endpoints (`METHOD /path` – purpose)

Endpoint	Purpose
`GET /`	Leaflet single-page application
`POST /api/analyze`	Evaluate a clicked point `{lat, lon}`
`GET /api/analyze/{id}`	Retrieve a past point analysis
`POST /api/analyze-area`	Best site inside `{bounds: min/max lat/lon}`
`POST /analyzeContour`	Best site on an uploaded KML/KMZ contour map
`GET /api/villages/search?q=`	Village geocoding (curated index + Nominatim)
`GET /api/villages/{id}/history`	Past analyses for a village
`GET /api/analyses/recent`	Twelve most recent analyses
`GET /api/health`	Status, storage mode, uptime, memory (used by the watchdog)

5.2 Frontend and Visualization

The interface (Fig. 2) is a single page with three modes: Point Analysis, Land Area Selection (drag a box or type its bounds) and Contour Map upload. Leaflet renders the satellite basemap; the result adds a pulsing pond marker whose popup lists the location, catchment area, expected water volume, depth and capacity; the catchment polygon, whose tooltip repeats the area and volume; and the analysed parcel as a dashed rectangle. The side panel shows metric cards, the sizing parameters, a Chart.js monsoon/non-monsoon rainfall chart and any warnings, with GeoJSON export and a printable summary.

5.3 Database and Storage

With PostgreSQL available, each point analysis is stored across five PostGIS tables (`village`, `analysis_request` with a POINT, `terrain_result` with the catchment GEOMETRY, `rainfall_result`, `pond_recommendation`) in SRID 4326, and consolidated responses are cached in `query_cache` under a key of coordinates rounded to three decimals (≈110 m) with a 7-day TTL. A 3 s connection check at startup selects this mode; otherwise (as on the evaluation server) history is kept in `data/analyses_history.json` (100 records) and the cache in a bounded in-memory map (500 entries).

6 CSD Themes and Topics Applied in the Project

Resource-aware admission control. The provided server reports 120 CPU cores but its cgroup allows one CPU and 512 MiB of memory. Two effects followed. First, NumPy, OpenBLAS and Numba each started about 120 threads for one CPU, and a contour analysis took 28–49 s; pinning their thread pools to one thread in the start script brought it to 0.8 s. Second, in our first stress test two concurrent contour analyses pushed the process past 512 MiB and the kernel OOM-killed the server (the watchdog restarted it). Since parallel heavy jobs cannot run faster on one CPU anyway, every heavy analysis now takes a single slot and others queue, receiving 503 if they wait more than 120 s; light requests never queue. The repeated test then passed with zero errors and a 330 MB peak.

Resilience to partial connectivity. The server can reach OpenTopography but not Open-Meteo or Nominatim. Instead of failing, or silently using different numbers, the app uses bundled real archive data within the demo region

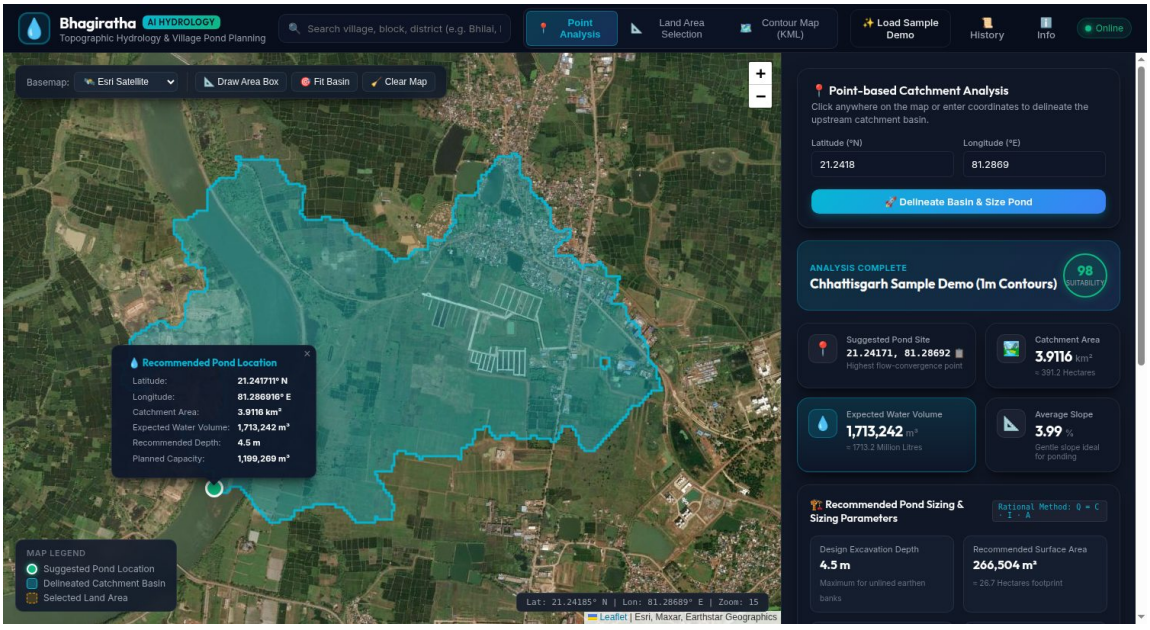


Fig. 2. Application interface after the sample contour analysis: catchment polygon, pond marker with its popup (location, catchment area, expected water volume, depth, capacity) and the results panel.

and a baseline elsewhere, tells the user, and uses a background probe and circuit breaker so no request waits on a blocked service. Results on the server therefore match results on an unrestricted network exactly.

7 Results and Evaluation

Contour survey. The sample map (1,355 contours, 267–298 m, $\approx 3.2 \times 2.6$ km near Durg) gives a site at 21.24171°N, 81.28692°E; a 3.9116 km^2 catchment with mean slope 3.99%; 1,415.2 mm annual and 1,251.4 mm monsoon rainfall; runoff $Q = 0.35 \times 1.2514 \times 3.9116 \times 10^6 = 1{,}713{,}242$ m^3; and a 4.5 m deep pond of 266,504 m^2 holding 1,199,269 m^3 (suitability 98). **Land-area selection** (Fig. 3) of the parcel 21.235–21.255°N, 81.278–81.298°E on real SRTM data places the pond inside the parcel at 21.23611°N, 81.27806°E, with a 7.089 km^2 catchment extending upstream and 3,104,911 m^3 of expected runoff. No field measurements were available for comparison, so correctness was checked by consistency: the vector polygon's area matches the cell-count area (3.9114 vs. 3.9116 km^2), the polygon is valid and does not touch the map edge, and the hand-calculated Q above equals the system output.

7.1 Performance

Table 4 gives measured latencies. Stage times are medians of three runs on a development machine; stress results were measured on the evaluation server `stu72_sys3` with the load generator on the same container, which competes for the single CPU, so they are pessimistic.

Table 3. Mapping of core CSD themes/topics to their use in this project

CSD Theme/Topic	Where used in the project	Justification / design rationale
API Design (REST)	Nine JSON endpoints, Pydantic request/response models, OpenAPI at `/docs`	Typed contracts validate input at the edge and keep the frontend decoupled from the engines.
Load Balancing	Not used; single process with a heavy-job queue (admission control)	The server has one CPU and 512 MiB: extra instances would compete for the same CPU and memory. Queueing heavy jobs is the effective load control here; see the paragraph below.
Caching	Rounded-grid response cache (7-day TTL); bundled rainfall archive	Repeat point queries drop from ≈18 s to milliseconds, and external rate-limited APIs are called less.
Database Indexing / Query Optimization	PostGIS geometry columns, primary/foreign keys on `request_id`, keyed cache lookups	Spatial types store catchments natively; lookups are by primary key.
Concurrency / Asynchronous Processing	`asyncio` tasks run DEM download and rainfall query in parallel; CPU work in worker threads	The rainfall query is hidden behind the download, and `/api/health` stays responsive (p95 0.42 s) during heavy analyses.
Microservices vs. Monolith	Modular monolith	Raster grids would have to be serialised between services; one process is simpler to run on one container while modules stay separable.
Design Patterns	Circuit Breaker (rainfall), fallback chain (live → bundled → baseline), Adapter (KML and SRTM → one GeoTIFF interface), dependency injection (DB session)	Each isolates a varying part: an unreliable service, a data source, or storage availability.
Authentication / Authorization	Not implemented	Read-only analysis on the campus network; no user data is stored.
Containerization / Deployment	Campus container `stu72_sys3`; start script and watchdog; thread pools pinned to the CPU quota	Reproducible start, automatic restart within 60 s, and no thread oversubscription.
Error Handling and Resilience	Fallbacks with user-visible warnings, retries, circuit breaker, 503 on overload, watchdog	The service degrades gracefully instead of failing silently or crashing.
Algorithms and Complexity	Priority-Flood $O(N \log N)$; D8 and accumulation $O(N)$	Standard, provably correct hydrology; under 0.2 s for 3×10^4 cells.
Testing Strategy	28 automated tests plus a stress test (§7.2)	Regression checks on the formulas, file handling and failure modes.
Version Control / CI-CD	Git and GitHub; no CI pipeline	The test suite is run manually before each deployment.

7.2 Testing

Twenty-eight automated tests (`pytest`) cover KML/KMZ parsing (including malformed, empty and wrong-type files), contour interpolation, D8 delineation on synthetic and SRTM-format `int16` tiles, the Rational-method unit conversions, depth bounds, score continuity, parcel containment, pixel alignment, caching, the database layer, the rainfall fallback and circuit breaker, a blocked-network run that must reproduce the live result, and the heavy-job queue. A stress script (`scripts/stress_test.py`) produced Table 4.

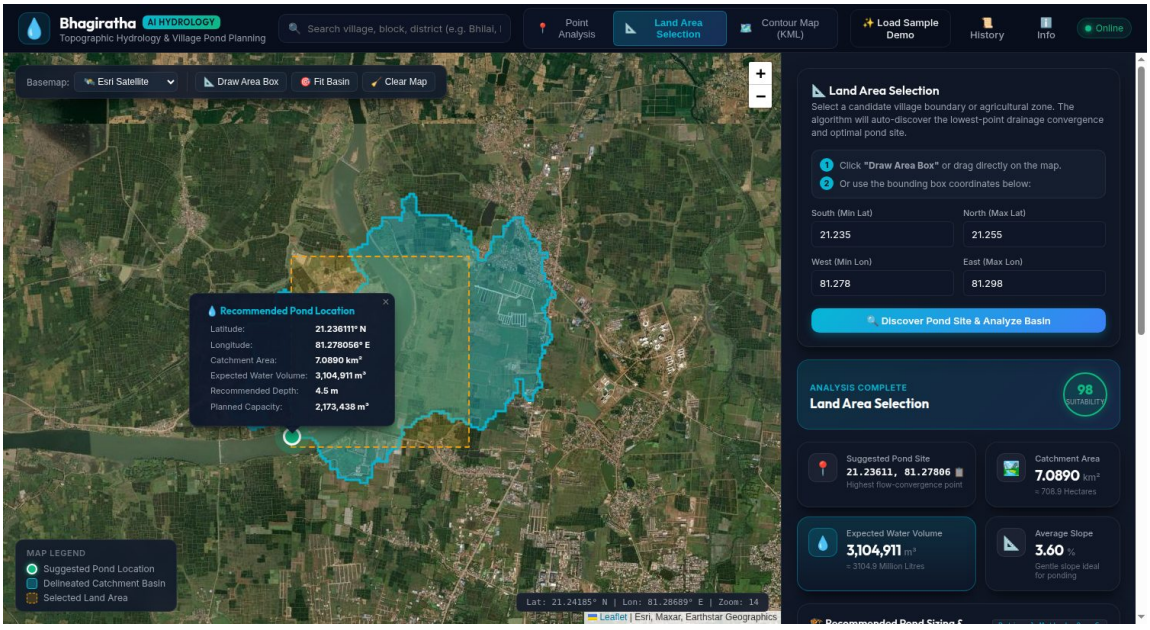


Fig. 3. Land-area selection: the analysed parcel (dashed orange), the suggested pond site inside it, the catchment extending upstream, and the popup with catchment area and expected water volume.

Table 4. Measured performance

Operation	Result
KML parse / interpolation / flow routing + catchment	0.33 s / 0.72 s / 0.16 s
Open-Meteo 10-year query / SRTM tile download	1.25 s / ≈12 s
Contour analysis end-to-end on the server (incl. first request)	0.7–0.9 s
Point analysis: first request / cached	17.5 s / p50 0.13 s
`/api/health`: 200 requests, 20 concurrent	0 errors, p95 0.28 s, 163 req/s
Cached point: 200 requests, 20 concurrent	0 errors, p95 0.42 s, 107 req/s
Contour, concurrency 1 / 2 / 4 (4 requests each)	0 errors; p50 0.84 / 1.69 / 7.10 s
Health latency during 4 concurrent contours	p95 0.42 s
Server memory idle / peak under load	291 MB / 330 MB (limit 512 MiB)

8 Discussion and Limitations

What worked: the pipeline is fast (under 1 s for contours), the results are consistent across data sources, and the service survives the server's CPU, memory and network limits. Measuring on the real server was decisive: the CPU oversubscription, the out-of-memory crash and the blocked rainfall service were all found by measurement, not assumed. **Limitations:** (i) On the sample map the maximum-convergence point lies in the Shivnath river channel. That is hydrologically correct, but it is a check-dam location, not a site for a dug pond; the method has no knowledge of existing water bodies, ownership, land use, soil or approvals, so results are screening recommendations. (ii) Retaining 70% of the runoff from 3.9 km^2 needs a 26.7 ha pond, far larger than a village pond; for large catchments the output is better read as the harvesting potential to split across several structures. (iii) SRTM's 30 m resolution misses channels

that the 1 m contour survey captures, and point results can depend on the tile extent. (iv) C is constant, and evaporation and seepage are not modelled. **Future work:** exclude OpenStreetMap water bodies (the river and four canals are already mapped there), rank several candidate sites, add land-use, soil and cadastral layers, and model the water balance.

9 AI Tool Usage Declaration

AI assistants were used during development: Google Antigravity and Anthropic Claude (Claude Code). Uses: debugging KML namespace handling and PySheds/Numba compatibility; a pre-submission audit that found and fixed defects (latitude-uncorrected D8, pixel misalignment, crashes on SRTM `int16` tiles, silent fallbacks, the out-of-memory crash); deployment hardening and stress testing on the campus server; writing tests; and drafting and reformatting this report into the course template. All AI-assisted code and text was reviewed, understood and modified by the author before submission.

A Source Code and Repository

Repository: https://github.com/RahulSeera/pond-planning-telemetry. Live application (campus network): http://10.1.75.53:3246/. Demo video: https://youtu.be/vMweMjVhEAE. Structure: `app/main.py` (API and orchestration), `app/heavy_jobs.py` (admission control), `app/modules/` (terrain, catchment, contour, rainfall, recommendation, village search), `app/db/` (PostGIS models, cache), `app/static/` (frontend), `data/` (bundled rainfall, history), `scripts/` (start, watchdog, stress test, rainfall builder), `tests/` (28 tests), `report/` (this report and demo guides).

References

[1] R. Barnes, C. Lehman, and D. Mulla. 2014. Priority-flood: An optimal depression-filling and watershed-labeling algorithm for digital elevation models. *Computers & Geosciences* 62 (2014), 117–127.

[2] J. F. O'Callaghan and D. M. Mark. 1984. The extraction of drainage networks from digital elevation data. *Computer Vision, Graphics, and Image Processing* 28, 3 (1984), 323–344.

[3] V. T. Chow, D. R. Maidment, and L. W. Mays. 1988. *Applied Hydrology*. McGraw-Hill, New York.